

Nombre y apellido: LU: DNI:
E-mail: # Hojas: Firma:

Poner nombre, fecha y firma en cada hoja. Justificar todas las respuestas.

1.a	1.b	1.c	1.d	1.e	2.a	2.b	2.c	2.d	3.a	3.b	3.c	Nota

Ejercicio 1 (Programación funcional). El tipo algebraico `Trie a` describe los árboles de tipo *trie* (del inglés *reTRIEval*), que se puede usar en forma *naive* para representar conjuntos o diccionarios.

```
data Trie a = TrieNode (Maybe a) [(Char, Trie a)]
```

El camino desde la raíz hasta un nodo determina una cadena. Si el nodo contiene un valor de la forma `Just v`, entonces esa cadena representa una palabra del conjunto o una clave definida del diccionario. Si contiene `Nothing`, entonces la cadena no está definida como palabra o clave, aunque puede ser prefijo de otras cadenas.

Decimos que la representación es *naive*, porque la lista de hijos puede contener más de una aparición del mismo carácter. Por eso, una misma cadena puede estar representada por más de un camino dentro del *trie*.

Por ejemplo:

```
t :: Trie Bool
t =
  TrieNode Nothing
    [ ('a', TrieNode (Just True) [])
    , ('b', TrieNode Nothing
      [ ('a', TrieNode (Just True)
        [ ('d', TrieNode Nothing []) ])
      ])
    , ('c', TrieNode (Just True) [])
    ]
```

- Dar el tipo y definir una función `foldTrie`, que abstraiga el esquema de recursión estructural sobre los *tries*.
- Sin usar recursión explícita, definir la función `palabras :: Trie a -> [[Char]]` que, dado un *trie*, devuelva todas las palabras incluidas en él. Una palabra está incluida si y solo si el nodo alcanzado por ese camino desde la raíz contiene un valor de la forma `Just _`, y esto incluye a la raíz, lo que implica que se incluye a la palabra vacía. No se deben incluir palabras repetidas. Por ejemplo, dado el *trie* anterior:
`palabras t ~> ["", "ba", "c"]`
- Sin usar recursión explícita, dar el tipo y definir una función `mapTrie` que aplique una función dada a todos los valores de un *trie*.
- Se quiere definir una función `conLongitud :: Trie a -> Trie (a, Int)` que asocie a cada valor almacenado la longitud de la palabra correspondiente. ¿Puede definirse usando únicamente `mapTrie`? Justificar. En caso negativo, definir una función más general que permita implementarla.

Ejercicio 2 (Programación Lógica y Resolución).

Una *estructura arbórea* es o bien el símbolo `o` o bien una lista de estructuras arbóreas. Por ejemplo, las siguientes son estructuras arbóreas válidas:

`o` `[]` `[o,o]` `[[o,o],o,[o,o]]` `[o,[o,o,[o,[],[o,o]],o],[o,[]]]`

- Definir en Prolog un predicado `estructuraArborea(-E)` que genere todas las posibles estructuras arbóreas.
- Pasar la base de conocimientos a formal clausal.
- Dar un ejemplo de consulta que, bajo la estrategia usual de Prolog, no finalice en alguna situación.
- Mostrar que, usando Resolución General, puede responderse la consulta anterior. Explicar la diferencia con Prolog.

(El examen sigue al dorso →)

Ejercicio 3 (Cálculo- λ). Se extiende el cálculo- λ con un constructor de términos $M ::= \dots \mid M \bowtie N$. No se extienden los conjuntos de tipos ni el conjunto de valores. Se agregan las siguientes reglas de tipado y reducción:

$$\frac{\Gamma \vdash M : \tau \quad \Gamma \vdash N : \tau}{\Gamma \vdash M \bowtie N : \tau} \text{T-}\bowtie \quad \frac{M \rightarrow M'}{M \bowtie N \rightarrow M' \bowtie N} \text{E-}\bowtie_1 \quad \frac{M \rightarrow M'}{V \bowtie M \rightarrow V \bowtie M'} \text{E-}\bowtie_2$$

$$\frac{}{V_1 \bowtie V_2 \rightarrow V_1} \text{E-SELECT}_1 \quad \frac{}{V_1 \bowtie V_2 \rightarrow V_2} \text{E-SELECT}_2$$

Observar que se pierde el determinismo, ya que por ejemplo $\text{true} \bowtie \text{false} \rightarrow \text{true}$ y $\text{true} \bowtie \text{false} \rightarrow \text{false}$.

- Decidir si es verdadera o falsa y justificar: existe un valor V tal que $V \text{ true} \rightarrow^* \text{true}$ y también $V \text{ true} \rightarrow^* \text{false}$.
- Si M es un término cerrado y tipable, definimos $\mathcal{E}(M)$ como el conjunto de los valores V tales que $M \rightarrow^* V$. Por ejemplo, $\mathcal{E}(\text{true} \bowtie (\text{true} \bowtie \text{false})) = \{\text{true}, \text{false}\}$. Suponiendo que $\mathcal{E}(M)$ tiene un elemento y $M : \text{bool}$, definir M' tal que $\mathcal{E}(M')$ tenga dos elementos.
- Decidir si es verdadera o falsa y justificar: existe un término cerrado y tipable M tal que $\mathcal{E}(M (M (\text{true} \bowtie \text{false})))$ es un conjunto de 8 elementos.